

Mail Tracker V2

Packaged Project Guide

Unchanged module layout · richer modules · PEP 621 / PEP 735 packaging

Version 2.1.0 · August 2026

This document describes the transformed Mail Tracker V2 tree: same modules/ layout, installable package metadata, dependency groups, and a short map of packaging topics (uv lock, resolution markers, forking, PubGrub vs pip vs Poetry, conflict resolution).

1. Project structure

Layout is intentionally unchanged from the original Mail Tracker V2 shape. Orchestration lives in a package module; the public API is re-exported from the package root.

```
mail_tracker_v2_packaged/
■■■ pyproject.toml
■■■ README.md
■■■ tests/
■   ■■■ test_pipeline.py
■■■ mail_tracker_v2/
    ■■■ __init__.py           # public API
    ■■■ pipeline.py          # end-to-end (was main_*.py)
    ■■■ modules/
        ■■■ email_identity.py
        ■■■ email_parser.py
        ■■■ email_semantic.py
        ■■■ email_project_mapper.py
        ■■■ email_dept_mapper.py
        ■■■ email_sla.py
        ■■■ email_lifeline.py
        ■■■ email_workflow.py
```

2. Pipeline flow

```
raw_email
→ parse_email
→ map_project / map_dept
→ generate_uid_v2
→ semantic_classify (category + risk + confidence)
→ compute_sla
→ build_lifeline
→ workflow_task
→ unified result dict
```

Sample subject P2382 risk on validation schedule resolves to category **Risk**, priority **P0**, SLA **4h**, dept **RD**.

3. Modules (richer responsibilities)

Module	Role
email_identity	SHA-256 UID v2 from sender, receiver, subject, timestamp, project, dept
email_parser	Canonical schema; list receivers; timestamp normalize
email_semantic	Ordered rule matrix → category, risk_level, keywords, confidence
email_project_mapper	Known-list match then regex (Pxxxx / PROJ-...)
email_dept_mapper	Dept from sender/receiver local-part patterns
email_sla	Policy table: Risk 4h ... Action 24h
email_lifeline	Persistent tracker record + optional extra fields
email_workflow	Task type, team routing, P0–P3 from risk

4. Packaging (PEP 621 + PEP 735)

Build backend: **Hatchling**. Runtime uses the standard library only. Dev tools live in [dependency-groups] (not published to PyPI).

4.1 Dependency groups vs extras

Kind	Table	Audience	Example install
Groups	[dependency-groups]	Developers	uv sync --group test
Extras	[project.optional-dependencies]	Package users	pip install pkg[viz]

4.2 Groups in this project

```
[dependency-groups]
dev = [{include-group = "lint"}, {include-group = "test"}]
lint = ["ruff>=0.8", "mypy>=1.13"]
test = ["pytest>=8.0", "pytest-cov>=5.0"]
```

```
[tool.uv]
default-groups = ["dev"]
```

4.3 Commands

```
uv sync          # project + dev group
uv run mail-tracker  # CLI sample
uv run pytest
uv sync --only-group test  # lean CI
uv lock          # write universal uv.lock
```

5. Resolver map (investigation summary)

Tool	Algorithm	Lock scope
pip	resolvelib backtracking	Mostly current environment
Poetry	PubGrub (Python)	Universal-style poetry.lock
uv	PubGrub (Rust) + marker forks	Universal uv.lock

uv forking: when the same package needs different pins under disjoint markers (e.g. Python version), the resolver splits into forks, solves each, and records resolution-markers in `uv.lock`. **--fork-strategy requires-python** prefers newest per Python band; **fewest** minimizes distinct versions across the range.

Conflict resolution (PubGrub): on version conflict, learn an incompatibility clause, backjump to the root cause, and prune the search. Marker conflicts become forks instead of hard failures. Explicit `tool.uv.conflicts` separates incompatible extras/groups.

6. Quick API

```
from mail_tracker_v2 import mail_tracker_v2

result = mail_tracker_v2({
    "sender": "rd_lead@example.com",
    "receiver": "pm@example.com",
    "subject": "P2382 risk on validation schedule",
    "body": "We see potential risk and delay on validation phase.",
    "timestamp": "2026-08-25 16:50",
})
# result["UID"], ["Semantic"], ["Project"], ["Dept"],
# ["SLA"], ["Lifeline"], ["Task"]
```

7. Artifacts

- `mail_tracker_v2_packaged/` — source tree
- `mail_tracker_v2_packaged.zip` — archive
- `updated_code_all.zip` — earlier NPL + tracker upgrades
- This PDF — structure + packaging guide

Mail Tracker V2 · Packaged Project Guide · MIT